GRAPHQL I DATALOADER

Клієнт замовляє форму відповіді — хто платить?

Заняття №17

Node.js Pro

Лектор
Кирило Орлов

r_d

СКЛАДНІСТЬ НЕ ЗНИКАЄ — ВОНА ПЕРЕЇЖДЖАЄ

REST: біль у КЛІЄНТА — waterfall запитів або зайві кілобайти

GraphQL: біль переїжджає на СЕРВЕР — у resolvers...

...а звідти в БАЗУ: N+1, 42 SQL на один HTTP-запит

DataLoader: біль береться ПІД КОНТРОЛЬ — 4 SQL замість 42

Рахунок за переїзд: HTTP-кеш і 404 лишаються в REST

Кожен переїзд сьогодні — наживо в SQL-лозі

ПЛАН ЗАНЯТТЯ



- Біль REST: under- і over-fetching у байтах
- Code-first: схема виростає з декораторів
- Resolvers і граф одним запитом
- N+1 наживо в SQL-лозі → DataLoader
- Мутації + валідація тими самими DTO
- Чесне порівняння з REST + межі GraphQL
- Три живі демо: A · B · C

ДВА CURL — ОБИДВА ПОГАНІ

```
$ curl -s -o /dev/null -w '%{size_download}' localhost:21700/orders
812     ← байтів; але імен клієнтів ТУТ НЕМАЄ — а екрану вони треба

$ curl -s -o /dev/null -w '%{size_download}' localhost:21700/orders-full
4588    ← байтів «про всяк випадок»; екрану треба ~800
        лог сервера: разом SQL-запитів на цей HTTP-запит: 42

under-fetching або over-fetching. Третього REST не пропонує.
```



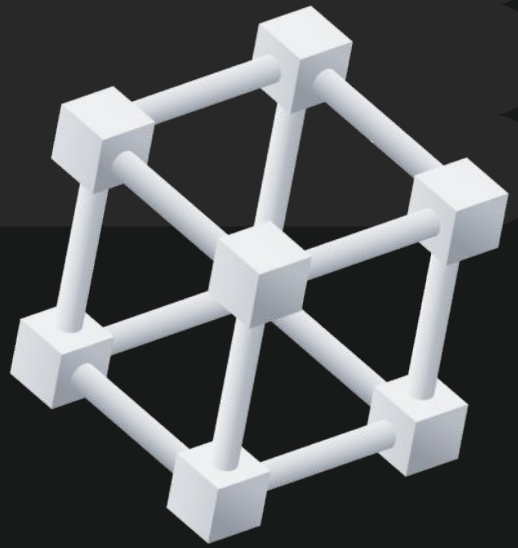
ЕКРАНУ ТРЕБА ІМЕНА — REST ПРОПОНУЄ ДВА ПОГАНІ ШЛЯХИ

/orders → 812 байтів, "customerId": 1 — а ім’я добирай ще десятьма GET /customers/:id (waterfall)

/orders-full → 4588 байтів і 42 SQL: сервер зібрав УСЕ, бо не знає, що треба екрану

Третій шлях — /orders-for-mobile-v2 під кожен екран: це вже меню, а не API

N+1 уже живе в цьому REST — просто його ніхто не бачить



ОДИН ENDPOINT · ТИПІЗОВАНА СХЕМА · ФОРМУ ОБИРАЄ КЛІЄНТ

```
схема (контракт)      запит          відповідь
type Order {          { orders {     { "data": { "orders": [
  id: Int!                id             { "id": 1,
  status: String!         status           "status": "new" },
  customer: Customer! } }                … ] } }
}
```

Запит ДЗЕРКАЛИТЬ відповідь: чого не попросив — того не існує
SDL — мова опису схеми; schema-first = SDL руками,
code-first = SDL генерується



```
// CODE-FIRST: СХЕМА ВИРОСТАЄ З ДЕКОРАТОРІВ
// src/shared/types.ts — SDL ніхто не пише руками:
// він згенерується в schema.gql

@ObjectType({ description: 'Замовлення' })
export class Order {
  @Field(() => Int)
  id!: number;

  @Field()
  status!: string;

  @Field(() => GraphQLISODateTime)
  createdAt!: Date;

  // FK у схему НЕ потрапляє — @Field на ньому немає
  customerId!: number;
}

// Ті самі декоратори + reflect-metadata, що в L#6 —
// метадані стають схемою
```



```
// RESOLVER I autoSchemaFile

@Resolver(() => Order)
class OrdersResolver {
  @Query(() => [Order], { description: 'Усі замовлення' })
  orders(): Promise<Order[]> {
    return repo.allOrders();
  }
}

GraphQLModule.forRoot<ApolloDriverConfig>({
  driver: ApolloDriver,
  autoSchemaFile: join(process.cwd(), 'schema.gql'), // сюди ляже SDL
  sortSchema: true,                                  // стабільний диф у git
})

// Схема — артефакт збірки, не вихідник.
// Schema-first у Nest теж є — ми обрали генерацію
```



🔴 ДЕМО A · БІЛЬ REST → ПЕРША СХЕМА

A1 — /orders: 812 байтів, імен немає → A2 — /orders-full: 4588 байтів, у лозі — 42 SQL → A3 — step2: schema.gql генерується CAM із декораторів

A4 — { orders { id status } } → 293 байти (10 замовлень) → Поле phone → GRAPHQL_VALIDATION_FAILED і 0 SQL: схема відсікла ДО бази



```
// ГРАФ ОДНИМ ЗАПИТОМ: @ResolveField

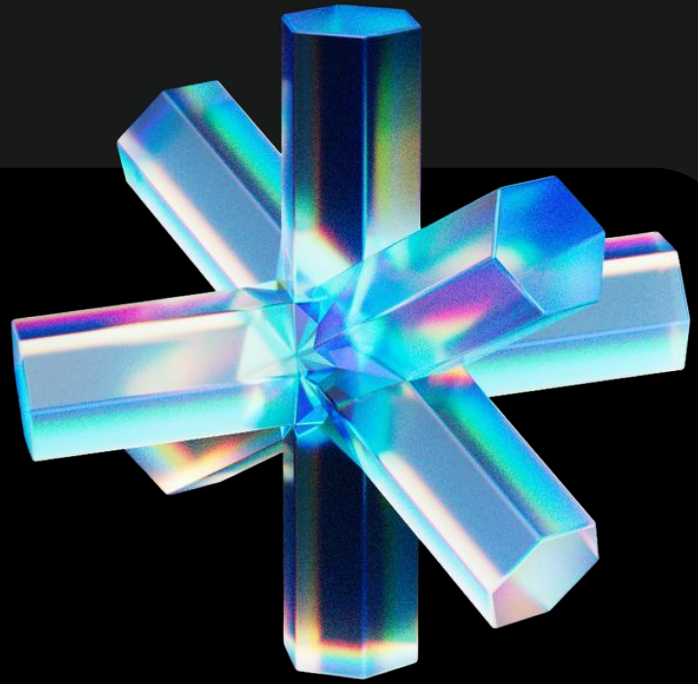
@Resolver(() => Order)
class OrdersResolver {
  @Query(() => [Order])
  orders() { return repo.allOrders(); }

  // викликається НА КОЖЕН Order у відповіді: 10 замовлень = 10 викликів
  @ResolveField(() => Customer)
  customer(@Parent() order: Order) {
    return repo.customerById(order.customerId);
  }

  @ResolveField(() => [OrderItem])
  items(@Parent() order: Order) {
    return repo.itemsByOrderId(order.id);
  }
}

// Скалярні поля віддає trivial resolver (дефолтний) — код не пишеться
```

N+1 У ЛОЗІ: 1 + 10 = 11.
ПОВНИЙ ГРАФ — 42



```
{ orders { id customer { name } } }

SQL#1  SELECT * FROM orders
SQL#2  SELECT * FROM customers WHERE id = $1
SQL#3  SELECT * FROM customers WHERE id = $1
…              ще вісім таких самих
→ разом SQL-запитів на цей HTTP-запит: 11

{ orders { id customer { name }
           items { qty product { title } } } }
→ разом SQL-запитів на цей HTTP-запит: 42

Рівно стільки ж, скільки «жирний» REST з кроку 1:
GraphQL N+1 не СТВОРИВ — УСПАДКУВАВ.
Але дав одне місце для фіксу
```

АНАЛОГІЯ: ОФІЦІАНТ І КУХНЯ

ЧАСТИНА GRAPHQL	АНАЛОГІЯ (РЕСТОРАН)
• REST • GraphQL зі схемою • N+1 • DataLoader	• комплексні обіди: № 1 або № 2, бери як є • офіціант з меню-схемою: страву збираєш сам, у межах меню • наївний офіціант бігає на кухню за КОЖНИМ інгредієнтом окремо • офіціант збирає замовлення з усього столу і йде РАЗ

ДЕ ЛАМАЄТЬСЯ: офіціант може чекати весь вечір — DataLoader чекає лише ОДИН tick event loop. І в ресторані платить гість — тут за свободу клієнта платить КУХНЯ (база).

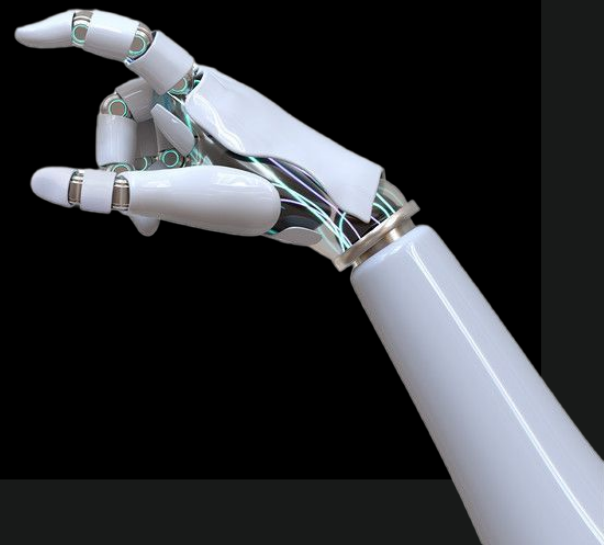


DATALOADER: УСІ .LOAD() ОДНОГО TICK — В ОДИН BATCH

```
10 × loaders.customerById.load(id)  ← з resolvers,
        │                             в ОДНОМУ tick
        └─ дедуплікація: 10 ключів → 8 унікальних
   BATCH customers ← [1, 2, 3, 4, 5, 6, 7, 8]
   SQL#2  SELECT * FROM customers WHERE id = ANY($1)

→ разом: 2 (було 11) · повний граф: 4 (було 42)

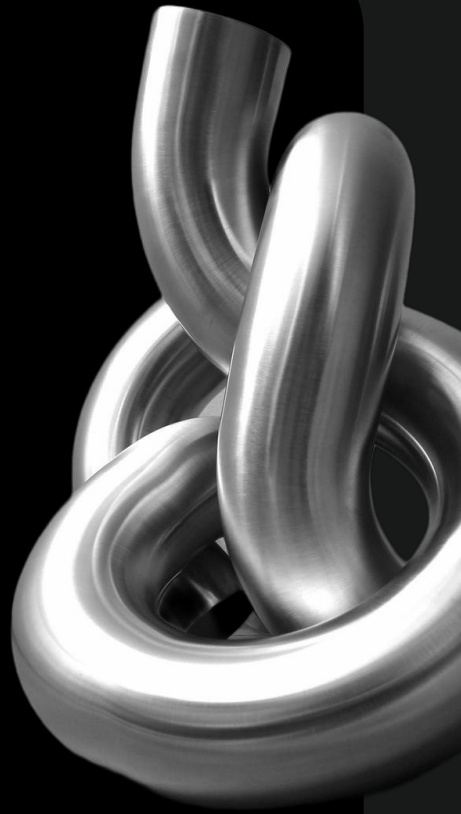
Три речі за одним фасадом:
batching · дедуплікація · кеш повторів load(id)
```



```
// КОНТРАКТ BATCH-ФУНКЦІЇ: ПОРУШИШ — ЗЛАМАЄТЬСЯ МОВЧКИ

// src/step04-dataloader/loaders.ts
customerById: new DataLoader(async (ids) => {
  const rows = await repo.customersByIds(ids);  // WHERE id = ANY($1)
  const byId = new Map(rows.map((c) => [c.id, c]));
  return ids.map((id) => byId.get(id) ?? null);
  //     ↑ ТОЙ САМИЙ порядок і ТА САМА довжина, що в ids
}),

// 1. Стільки ж результатів, скільки ключів
// 2. У тому самому порядку — ANY() порядку НЕ гарантує,
//    мапимо через Map руками
// 3. Немає значення → null НА ЙОГО МІСЦІ, а не «пропустити»
```



```
// LOADER ЖИВЕ ОДИН HTTP-ЗАПИТ — І ЦЕ НЕ ВИПАДКОВІСТЬ

GraphQLModule.forRoot<ApolloDriverConfig>({
  // …
  context: () => ({ loaders: createLoaders() }), // НА КОЖЕН запит
})

// DataLoader ще й КЕШУЄ: повторний load(3) — з пам'яті, без SQL
// Глобальний loader = кеш переживає запит →
//    клієнт бачить чужі застарілі дані
// context — те саме per-request місце, де в L#8 жив requestId
```

🔴 ДЕМО B · N+1 НАЖИВО → DATALOADER

B1

step3: { orders { id customer { name } } } → у лозі 11 SELECT

B2

повний граф → разом: 42

B3

step4, TI CAMI запити → BATCH customers ← [1, 2, 3, 4, 5, 6, 7, 8]

B4 — повний граф → разом: 4 (було 42)
SQL змінив форму: WHERE id = ANY($1). Жоден resolver не змінився.



МУТАЦІЯ: ТІ САМІ DTO І VALIDATIONPIPE, ЩО В L#7

app.useGlobalPipes(new ValidationPipe(...)) — буквально той самий рядок, що в L#7

```
@InputType()
class OrderItemInput {
  @Field(() => Int)
  @IsInt() @Min(1)
  @Max(100, { message: 'qty не може перевищувати 100' })
  qty!: number;
}

@Mutation(() => Order, { description: 'Створити замовлення' })
async createOrder(
  @Args('input') input: CreateOrderInput,
): Promise<Order> {
  const customer = await repo.customerById(input.customerId);
  if (!customer)
    throw new NotFoundException(`customer ${input.customerId} not found`);
  return repo.createOrder(input.customerId, input.items);
  // BEGIN → INSERT → COMMIT
}
```



У GRAPHQL НЕМАЄ 404

```
qty: 500          → HTTP 200 · errors[]: "items.0.qty не може
                    перевищувати 100" · SQL-запитів: 0
customerId: 99    → HTTP 200 · errors[]: "customer 99 not found"
успіх             → HTTP 200 · data.createOrder.id = 11

Моніторинг по статус-кодах СЛІПНЕ —
alerts будуй по errors[] у тілі
Stack traces Apollo друкує в extensions, поки NODE_ENV
не 'production'/'test' — у проді вимкнути
```



🔴 ДЕМО C · МУТАЦІЯ, ВАЛІДАЦІЯ І КОМУ ДІСТАЄТЬСЯ КЕШ

C1. мутація

createOrder → id: 11; у лозі BEGIN → INSERT → COMMIT на одному connection

C2. валідація

qty: 500 → повідомлення з @Max, 0 SQL, HTTP 200

C3. немає 404

customerId: 99 → «404», який приїхав з HTTP 200

C4. HTTP-кеш

GET /rest/orders: ETag + Cache-Control → повтор 304, 0 байтів

підсумок

POST /graphql — завжди 200 і повне тіло, обидва рази

РАХУНОК: ЩО REST ДАЄ ЗАДАРМА

REST	GraphQL
GET /rest/orders	POST /graphql
ETag: W/"37f-…"	— (тіло щоразу повне)
Cache-Control: public, max-age=60	—
повтор: 304 · 0 байтів	повтор: 200 · повне тіло
404/400/500 видно моніторингу	помилки в errors[] при 200

Кешування в GraphQL БУДУЮТЬ:
persisted queries · кеш полів · CDN-шар



КОЛИ GRAPHQL, А КОЛИ REST

GraphQL — коли	REST — коли
багато РІЗНИХ клієнтів/екранів над одними даними (web + mobile + партнери)	CRUD з одним-двома клієнтами, форма відповіді стабільна
дані — граф, клієнти постійно просять нові комбінації полів	потрібні HTTP-кеш/CDN задарма і статуси для моніторингу
кілька команд над одним графом → Federation v2 (наступний слайд)	Жодне «завжди»: рішення — по клієнтах і даних, не по моді

АЛЬТЕРНАТИВИ
tRPC — type safety без схеми, коли клієнт і сервер в одному TS-репо. gRPC-Web — бінарні контракти protobuf до браузера (докладно — L#21).

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

ТЕМА

- Subscriptions — третій вид операцій
- Apollo Federation v2
- Persisted queries · depth/complexity limits · introspection off
- Schema-first детально

ДЕ ВОНА ЖИВЕ

- push через graphql-ws поверх WebSocket; транспорт розберемо в L#18 (старий subscriptions-transport-ws — deprecated)
- один граф з кількох subgraph-сервісів, коли графом володіють кілька команд; сьогодні — лише щоб термін не лякав
- у проді — стартові точки в матеріалах LMS
- у Nest є, ми свідомо обрали code-first

ПІДСУМКИ

ПЕРЕЇЗД

Складність переїжджає: клієнт (812/4588 байтів) → сервер → база (42 SQL) → під контроль (4)

DATALOADER

@ResolveField × об’єкт = N+1; DataLoader: batch одного tick + контракт batch-функції

ДАЛІ

У GraphQL немає 404 і безкоштовного HTTP-кешу — плати свідомо. Далі — L#18: subscriptions їздять через WebSocket

Тест #17

Тест #17: 3 бали · 6 питань · 1 спроба (ДЗ немає)

CODE-FIRST

Code-first: схема з тих самих декораторів, що L#6; валідація — ті самі DTO, що L#7

Q&A



r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ